

References and string input

CSC103 Programming Fundamentals / Lecture 12

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Read a student age from one line and full name from the next. Explain why `next()` is insufficient for a name containing spaces.

Read the age as an integer token.

Learning objectives

- Distinguish values from references
- Explain aliasing and null
- Read tokens and complete lines

CLO-2

Primitive and reference variables

Objects and null

String values are immutable

Token input and line input

Primitive and reference variables

- A primitive variable holds a primitive value.
- A reference variable holds a reference to an object or null.
- Copying a reference can make two variables refer to one object.

Example

```
int count = 3;  
String name = "Amina";  
String sameName = name;
```

Objects and null

- new requests construction of an object.
- null means a reference does not refer to an object.
- Calling an instance method through null throws NullPointerException.

Example

```
String text = null;  
boolean missing = text == null;
```

String values are immutable

- A String object represents text.
- Operations such as concatenation produce a result without changing the original String object.
- Reassignment changes which result a variable refers to.

Example

```
String a = "CS";  
String b = a;  
a = a + "C103";  
// b still refers to "CS"
```

Token input and line input

- `next` reads a token separated by whitespace.
- `nextLine` reads the rest of the current line.
- After `nextInt`, the pending line ending can yield an empty `nextLine` result.

Example

```
int age = in.nextInt();  
in.nextLine(); // finish the numeric line  
String fullName = in.nextLine();
```

A token read leaves the line ending

- `nextInt` consumes the integer token but usually leaves its following line separator.
- The next `nextLine` reads the remainder of that same line, which may be empty.
- When the full name belongs on the next line, finish the numeric line first.

Example

```
Input:  
20  
Amina Khan
```

```
The first nextLine after nextInt finishes line 1.
```

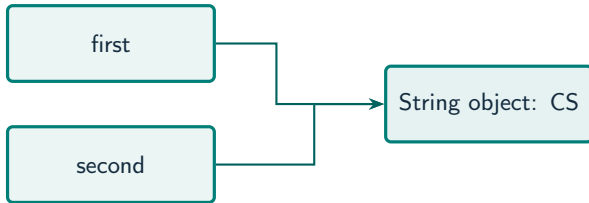
Aliasing differs from reassignment

- Two reference variables can identify the same object.
- Reassigning one variable changes only that variable's stored reference.
- String immutability means a transformation returns text rather than editing the original String.

Example

```
String a = "CS";  
String b = a;  
a = "Math";  
// b still denotes "CS"
```

Two variables can refer to the same string



`first + 103` gives a new result

What to notice

Before concatenation both references identify CS. Reassigning `first` leaves `second` unchanged.

Key distinctions

Value	Meaning	Safe length call?
null	No String object	No
""	An empty String	Yes, returns 0
"Ali"	A String with basic Latin text	Yes, returns 3

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
String first = "CS";  
String second = first;  
first = first + "C103";  
System.out.println(first);  
System.out.println(second);
```

First example: result

CSC103

CS

The second variable still refers to the original text.

String values are immutable

Token input and line input

Guided practice

Read a student age from one line and full name from the next. Explain why `next()` is insufficient for a name containing spaces.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Read the age as an integer token.
- Consume the remainder of that line.
- Read the next complete line as the name, preserving spaces.

Sample console input
20
Amina Khan

Complete solution

```
public class Solution12 {  
    public static void main(String[] args) throws Exception {  
        java.util.Scanner in = new java.util.Scanner(System.in);  
        int age = in.nextInt();  
        in.nextLine();  
        String fullName = in.nextLine();  
        System.out.println(fullName);  
        System.out.println(age);  
    }  
}
```

Execution trace

Operation	What it reads	Stored result
nextInt()	20	age = 20
first nextLine()	Remainder of numeric line	Discarded empty text
second nextLine()	Amina Khan	fullName includes the space

Follow the changing state in execution order.

Solution result

Amina Khan
20

Why this result follows
Read the next complete line as the
name, preserving spaces.

A common incorrect approach

```
String name = null;  
System.out.println(name.length());
```

Example

Calling `length` through `null` throws `NullPointerException`. Check `name != null` or ensure a valid input value first.

Boundary and failure checks

Check the stated input assumptions.

Build a profile input program with an integer ID and a full name on separate lines. Test a name with two spaces.

Read age with `nextInt`, finish that line with `nextLine`, then read the name with `nextLine`. `next` would return only the first token.

Check your understanding

Is "" the same as null? What happens to b after a="new" when b previously received a?

Write a prediction and explain the rule that supports it.

Answer and explanation

No. Empty text is a String object with length zero.
Reassigning a does not reassign b.

Calling length through null throws
NullPointerException. Check name
!= null or ensure a valid input value
first.

Compare cases and predict outcomes

Step	first refers to	second refers to
second = first	CS	CS
first = first + "103"	CSC103	CS
Print second	CSC103	CS is printed

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Predict the output after assigning `a="PF"`, `b=a`, and `a=a+"1"`. Explain why `b` does not change.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
String a = "PF";  
String b = a;  
a = a + "1";  
System.out.println(a);  
System.out.println(b);
```

- The output lines are PF1 and PF.
- Concatenation does not modify the original immutable String.
- Assignment changes the reference stored in a.

Ordering two city names

- Read full lines so a city name may contain spaces.
- `compareTo` returns a negative value, zero, or a positive value; its sign is what matters.
- This comparison is case-sensitive lexicographic ordering, not locale-aware dictionary collation.

Source: [supplied ISB campus slides](#). See [speaker notes](#) and [ISB_Source_Map.md](#).

Worked problem: Ordering two city names

Task

Read Islamabad and Sahiwal on separate lines and print them in lexicographic order.

Input: Islamabad / Sahiwal

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture12 {  
    public static void main(String[] args) throws Exception {  
        java.util.Scanner in = new java.util.Scanner(System.in);  
        String first = in.nextLine();  
        String second = in.nextLine();  
        if (first.compareTo(second) <= 0) {  
            System.out.println(first + " | " + second);  
        } else {  
            System.out.println(second + " | " + first);  
        }  
    }  
}
```

Worked trace and expected result

First / second	Comparison sign	Printed order
Islamabad / Sahiwal	Negative	Islamabad, Sahiwal
Sahiwal / Islamabad	Positive	Islamabad, Sahiwal
Sahiwal / Sahiwal	Zero	Both unchanged

Expected console output

```
Islamabad | Sahiwal
```

Extend the ISB example

Practice

Why is testing `compareTo(...) == -1` an incorrect general ordering test?

Explain your reasoning

Negative results are not restricted to -1. Use < 0 , $= 0$ or > 0 according to the relation.

Lecture summary

- values from references
- aliasing and null
- tokens and complete lines
- Read the age as an integer token.
- Consume the remainder of that line.
- Read the next complete line as the name, preserving spaces.

After-class practice

Build a profile input program with an integer ID and a full name on separate lines. Test a name with two spaces.

Syllabus reading

Deitel Ch 3

Next lecture: String processing